

CS201L: Artificial Intelligence Laboratory

Lab 9 – Regression : Linear Regression and Polynomial Curve Fitting

Indian Institute of Technology Dharwad

1 Problem Statement

You are given the dataset `EnergyEfficiency.csv`, which contains various building design parameters that affect the energy efficiency of residential buildings. The dataset is generated from simulations performed using Ecotect on different building configurations.

The objective of this task is to build regression models that learn the relationship between building structural features and the energy consumption of the building.

The goal is to predict the building's energy demand in terms of **Heating Load** using the available building design features.

2 Dataset Details

Input Variables:

- x_1 : Relative Compactness
- x_2 : Surface Area
- x_3 : Wall Area
- x_4 : Roof Area
- x_5 : Overall Height
- x_6 : Orientation
- x_7 : Glazing Area
- x_8 : Glazing Area Distribution
- x_9 : Cooling Load

Output Variables:

- y : Heating Load

Note: Different regression models will be built using:

- A single input feature
- The two most correlated input features
- All input features

These models will be compared based on their prediction performance for **Heating Load**.

3 Tasks

3.1 Data Splitting

Split the dataset into:

- 60% Training Data : `energyefficient_train.csv`
- 20% Validation Data : `energyefficient_val.csv`
- 20% Test Data : `energyefficient_test.csv`

3.2 Simple Linear Regression

1. Build a simple linear regression model to predict **Heating Load** (y) using **Relative Compactness** (x_1) as the independent variable.
2. Plot the **best fit line** on the training data.
 - (a) x-axis: Relative Compactness (x_1)
 - (b) y-axis: Heating Load (y)
3. Compute **RMSE** on training, validation, and test data.
4. Plot a **scatter plot** of actual vs predicted Heating Load on the test data.

3.3 Polynomial Curve Fitting

1. Build regression models with polynomial degrees:

$$p = 2, 3, 4, 5, 6, 7, 8, 9, 10, 11$$

to predict **Heating Load** (y) using **Relative Compactness** (x_1) as the independent variable.

2. Compute **RMSE** on the training and validation dataset for each degree.
3. Plot the **best fit curves** separately on training and validation dataset for each polynomial degree.

4. Plot the **Training and Validation RMSE vs Polynomial Degree**
 - (a) x-axis: Polynomial Degree
 - (b) y-axis: RMSE
5. Select the polynomial degree with the **lowest validation RMSE** and evaluate the selected model on the test data.
6. Compute the **RMSE** on the best model for the test dataset.
7. Plot a **scatter plot**: Actual vs Predicted Heating Load on the test data for the best polynomial model.

3.4 Multiple Linear Regression using All Input Features

1. Build a **Linear Regression** model using **all input features** to predict **Heating Load (y)**.
2. Use the trained model to predict the **Heating Load** for the **test dataset**.
3. Plot a **scatter plot of Actual vs Predicted Heating Load** for the test data.
 - (a) x-axis: Actual Heating Load
 - (b) y-axis: Predicted Heating Load

3.5 Correlation Analysis and Linear Regression

1. **Correlation Analysis**
 - (a) Compute the correlation between each input feature and the target variable **Heating Load (y)**.
 - (b) Calculate the correlation coefficient between all input variables ($x_1, x_2, x_3, \dots$) and the target variable y .
 - (c) Compare the correlation values.
 - (d) Identify the feature with the **highest absolute correlation** with the target variable.
2. **Simple Linear Regression using the Highest Correlated Feature**
 - (a) Let the selected feature be denoted as x_{best} .
 - (b) Train a linear regression model using x_{best} as the independent variable and y as the dependent variable.
 - (c) Plot the **best fit line** on the training data.
 - i. x-axis: Selected Feature (x_{best})
 - ii. y-axis: Heating Load (y)

- (d) Compute **RMSE** on training, validation, and test datasets.
- (e) Plot a **scatter plot of Actual vs Predicted Heating Load** for the test data.
 - i. x-axis: Actual Heating Load
 - ii. y-axis: Predicted Heating Load

3. Multiple Linear Regression using Top Two Correlated Features

- (a) Select the **two input features** that have the highest correlation scores with the target variable.
- (b) Build a **Linear Regression** model using these two selected features as independent variables.
- (c) Plot the **best fit plane** on the training data.
 - i. x-axis: Feature 1
 - ii. y-axis: Feature 2
 - iii. z-axis: Heating Load (y)
- (d) Use the model to predict **Heating Load** (y).
- (e) Compute **RMSE** on training, validation, and test datasets.
- (f) Plot a **scatter plot of Actual vs Predicted Heating Load** for the test data for the best polynomial model.
 - i. x-axis: Actual Heating Load
 - ii. y-axis: Predicted Heating Load

3.6 Model Comparison

After building the different regression models, compare their performance using the **Test RMSE** values.

1. Collect the **Test RMSE** values obtained from the following models:
 - Simple Linear Regression using Relative Compactness (x_1)
 - Best Polynomial Regression Model (selected using validation RMSE)
 - Simple Linear Regression using the Top Correlated Features
 - Multiple Linear Regression using the Top Two Correlated Features
 - Multiple Linear Regression using All Input Features
2. Create a **comparison table** that summarizes the Test RMSE for all the models.
3. The table should contain the following columns:
 - Model
 - Test RMSE
4. Use this table to **compare the performance of the different regression models**.

Sample Code Snippets for Regression Tasks

The following code blocks demonstrate how to implement the required correlation analysis, simple linear regression, and polynomial curve fitting using Python and scikit-learn.

1. **Correlation Analysis and Feature Selection:** Compute correlations with the target variable, identify the best feature, and prepare the splits.

```
1 import pandas as pd
2
3 # 1. Compute correlations with the target variable 'CO'
4 correlations = df.corr()['CO'].drop('CO')
5
6 # 2. Find the feature with the highest absolute correlation (
7     x_best)
8 x_best = correlations.abs().idxmax()
9 print(f"Feature with highest absolute correlation: {x_best}")
10
11 # 3. Define independent variables using x_best
12
13 X_train = train_df[[x_best]]
14 y_train = train_df['CO']
15
16 X_val = val_df[[x_best]]
17 y_val = val_df['CO']
18
19 X_test = test_df[[x_best]]
20 y_test = test_df['CO']
```

2. **Simple Linear Regression:** Train a linear model on the selected feature, plot the best fit line, and evaluate using RMSE.

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # 1. Build and train the simple linear regression model
7 lin_reg = LinearRegression()
8 lin_reg.fit(X_train, y_train)
9
10 # Generate predictions
11 y_train_pred = lin_reg.predict(X_train)
12 y_val_pred = lin_reg.predict(X_val)
13 y_test_pred = lin_reg.predict(X_test)
14
```

```

15 # 3. Compute RMSE for Training, Validation, and Test
16 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
17 rmse_val = np.sqrt(mean_squared_error(y_val, y_val_pred))
18 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
19 print(f"Train RMSE: {rmse_train:.4f} | Val RMSE: {rmse_val:.4f}
      | Test RMSE: {rmse_test:.4f}")

```

3. **Polynomial Curve Fitting:** Test different polynomial degrees, plot the RMSE comparisons, and evaluate the best degree on the test set.

```

1 from sklearn.preprocessing import PolynomialFeatures
2
3 degrees = [2, 3, 4, 5, 6]
4 train_rmses = []
5 val_rmses = []
6
7 # 1 & 2. Build models and compute RMSE for each degree
8 for p in degrees:
9     # Transform features
10    poly = PolynomialFeatures(degree=p)
11    X_train_poly = poly.fit_transform(X_train)
12    X_val_poly = poly.transform(X_val)
13
14    # Train model
15    poly_reg = LinearRegression()
16    poly_reg.fit(X_train_poly, y_train)
17
18    # Predict and calculate RMSE
19    train_rmse = np.sqrt(mean_squared_error(y_train, poly_reg.
20    predict(X_train_poly)))
21    val_rmse = np.sqrt(mean_squared_error(y_val, poly_reg.
22    predict(X_val_poly)))
23
24    train_rmses.append(train_rmse)
25    val_rmses.append(val_rmse)

```

4. **Plot For Linear Regression with Top Two Features:** Train a model on the top two correlated features and plot the 3D best fit plane.

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 from sklearn.linear_model import LinearRegression
4

```

```

5 # Assuming X_train_2d contains the two highest correlated
   features
6 # e.g., X_train_2d = train_df[['feature_1', 'feature_2']]
7
8 # 1. Train the model
9 model_2d = LinearRegression()
10 model_2d.fit(X_train_2d, y_train)
11
12 # 2. Set up the 3D plot
13 fig = plt.figure(figsize=(10, 8))
14 ax = fig.add_subplot(111, projection='3d')
15
16 # Extract features for easy plotting (assuming pandas DataFrame
   )
17 feat1 = X_train_2d.iloc[:, 0]
18 feat2 = X_train_2d.iloc[:, 1]
19
20 # 3. Scatter plot of the actual training data
21 ax.scatter(feat1, feat2, y_train, color='blue', alpha=0.6,
   label='Actual Data')
22
23 # 4. Create a meshgrid to plot the best fit plane
24 x1_range = np.linspace(feat1.min(), feat1.max(), 10)
25 x2_range = np.linspace(feat2.min(), feat2.max(), 10)
26 xx1, xx2 = np.meshgrid(x1_range, x2_range)
27
28 # Prepare the grid points for prediction
29 grid_points = np.c_[xx1.ravel(), xx2.ravel()]
30
31 # Predict Z values (Heating Load) for the plane and reshape
32 zz = model_2d.predict(grid_points).reshape(xx1.shape)
33
34 # 5. Plot the best fit plane
35 ax.plot_surface(xx1, xx2, zz, color='red', alpha=0.4)
36
37 # Labels and title
38 ax.set_xlabel('Feature 1')
39 ax.set_ylabel('Feature 2')
40 ax.set_zlabel('Heating Load (y)')
41 ax.set_title('3D Best Fit Plane')
42 plt.show()
43
44 # Compute RMSE for this model
45 y_test_pred_2d = model_2d.predict(X_test_2d)
46 rmse_test_2d = np.sqrt(mean_squared_error(y_test,
   y_test_pred_2d))
47 print(f"Test RMSE (2 features): {rmse_test_2d:.4f}")

```